

Typeform Builder

SDE Fullstack Assignment

Description

Build a functional clone of the Typeform application that replicates Typeform's design, user experience, and core form-building and form-filling workflows.

The platform should allow a creator to build forms with multiple question types via a drag-and-drop-style builder, publish them via a shareable link, collect responses through the signature one-question-at-a-time conversational experience, and view submitted results, all within the clean, focused interface of the original Typeform app.

Your implementation should visually and functionally feel like a modern Typeform. The two hardest and most important pieces are the builder and the polished, animated one-question-at-a-time respondent flow.

Note: Your application should match the original application in terms of UI and UX, look and feel should be exactly the same.

AI Tools Usage

You are allowed and encouraged to use AI tools such as ChatGPT, Claude, GitHub Copilot, Cursor, or any other AI assistant for development. Use AI as heavily as you like to move fast. However, you must understand every line of code you submit and be prepared to explain your implementation decisions during the evaluation interview.

Technical Stack

- Frontend: Next.js (TypeScript)
- Backend: Python with FastAPI / Django
- Database: SQLite (design your own schema)

Note: the respondent flow (public form fill) should be a real, shareable experience — no auth required to fill a published form.

Core Features (Must Have)

1. Form Builder

Recreate the Typeform builder.

- Create a form with a title and ordered list of questions
- Add, edit, reorder (drag-and-drop), and delete questions
- Question types: short text, long text, multiple choice, dropdown, email, number, yes/no, rating
- Per-question settings: required toggle, description/help text
- Live preview of the form

2. Form Management (CRUD)

- List of the creator's forms with status (draft/published) and response count
- Create, rename, duplicate, and delete forms
- Publish / unpublish, generating a shareable public link
- All form definitions must persist

3. Respondent Flow (the Typeform experience)

Implement the public form-filling experience.

- One question at a time, full-screen, with smooth transitions between questions
- Keyboard navigation (Enter/arrow to advance) and a progress indicator
- Client + server validation (required, email format, number, etc.)
- Submit stores the response; show a thank-you screen
- No login required to fill a published form

4. Results / Responses

- Per-form responses view (a table/list of submissions)
- View an individual response in full
- Basic summary stats per question (e.g. counts for choice questions)
- All responses must persist

5. Typeform Experience

The application should closely resemble the Typeform experience, including:

- The distinctive conversational, one-at-a-time fill UI with transitions
- Clean builder layout with live preview
- Forms, modals, and inline editing
- Notifications / toasts
- Settings placeholders (theme, thank-you screen)

The goal is to make the application feel like Typeform rather than a generic multi-field form.

Mocked / Placeholder Sections

The following can be present as placeholders (a simple “Coming Soon” is sufficient):

- Advanced logic jumps / branching (basic branching is a bonus)
- Integrations / webhooks
- Team collaboration & sharing
- Payment/file-upload question types
- Real creator authentication may be simplified (assume a default logged-in creator)

Bonus (Optional)

- Logic jumps / conditional branching
- Custom themes (colors, fonts, background)
- Export responses as CSV
- Partial-response tracking / completion rate

- File-upload question type
- Dark mode

Important Notes

- UI Design: your application should totally resemble Typeform's design. Study Typeform's UI carefully before starting.
- Sample Data: seed your database. Seed a couple of published forms with mixed question types and some existing responses so the app is immediately usable.
- Database Design: design your own database schema. This will be evaluated.
- README File: include setup instructions, tech stack used, architecture overview, database schema, and any assumptions made.
- Original Work: plagiarism from existing repositories will result in immediate disqualification.

Deliverables

- Source Code: a public GitHub repository containing frontend/ and backend/.
- Documentation: a README with setup instructions, architecture overview, database schema, and API overview.
- Demo: a hosted, working link.

Submission

- Upload your code to GitHub and ensure the repository is public.
- Deploy your application (Vercel, Netlify, Render, Railway, or any cloud service).
- Submit both the GitHub repository link and the deployed application link.

Evaluation Criteria

Criteria	What We Look For
Functionality	All core features working correctly, including the builder and the one-question-at-a-time respondent flow
UI/UX	Visual similarity to the original app's design and UX patterns
Database Design	Well-structured schema with proper relationships
Backend / API Design	Clean, sensible API design and architecture
Code Quality	Clean, readable, and well-organized code
Code Modularity	Proper separation of concerns, reusable components
Code Understanding	Ability to explain your code during evaluation

Timeline

Estimated effort: approximately 24 hours of work.

Submission Deadline: as communicated alongside this assignment.

Good luck!